

CLASE 12 PROGRAMADORES Y ON-LINE

Antes de comenzar, haga la siguiente pregunta a la IA y si considera necesario haga preguntas adicionales hasta comprender el tema:

¿En sistemas operativos que son las condiciones de competencia y por qué se producen?

Ejercicio 1

Solicita a la IA un ejemplo de uso de la interfaz Runnable de Java donde se creen dos hilos, y reutiliza el código para que ambos hilos accedan a una variable compartida de tipo int que inicialmente tiene un valor de cero:

```
static int variable_compartida = 0;
```

Verifique primero que ambos hilos pueden modificarla concurrentemente.

Verificado lo anterior haga que el método `run()` ejecute `n` veces (`n` se recibe como parámetro del programa) al método `modifica()`. El método `modifica()` se encarga de incrementar a `variable_compartida` en una unidad si se trata del hilo 1 o decrementarla en una unidad si se trata del hilo 2 como se muestra en el siguiente **pseudocódigo**:

```
public static void modifica(){
    id = Identificador_de_hilo();
    if(id == hilo1)
        var_compartida++;
    if(id == hilo2)
        var_compartida--;
}
```

Para esto debe preguntarle a la IA como se pueden asignar identificadores a los hilos.

Cuando ambos hilos hayan terminado de ejecutarse (pregunte a la IA sobre el método `join()` para asegurarse que ambos hilos se han ejecutado por completo), en la función principal deberá imprimir el valor de la variable compartida que en teoría debería ser cero.

Ejecute su programa varias veces incrementando `n` hasta poder visualizar las condiciones de competencia o condiciones de carrera. Es importante no imprimir nada en el programa para que el calendarizador de hilos opere de manera concurrente. Para asegurarse de que este ejercicio es correcto, pase conmigo para

revisar que efectivamente se dan las condiciones de competencia antes de realizar el ejercicio 2.

Ejercicio 2

Para el ejercicio anterior resuelva las condiciones de competencia haciendo que el método `modifica()` se ejecute sólo por un hilo a la vez con `synchronized` quedando:

```
public synchronized void modifica()
```

Ahora elabore un programa con tres hilos, y como variables compartidas tendremos un arreglo estático del tipo `char` denominado `pila` con diez elementos y una variable denominada `tope` del tipo `int`. El arreglo lo usaremos para simular una pila de diez elementos, donde la variable `tope` indica donde está el tope de la pila en un momento dado.

De los tres hilos uno debe funcionar como productor e introducir cada tiempo t_p aleatorio un elemento en la pila, y otro hilo debe funcionar como consumidor sacando cada tiempo t_c aleatorio un elemento de esta. El tercer hilo se encargará de borrar la pantalla e imprimir el contenido completo de la pila, así como el tope de la pila **cada vez que ésta sea modificada** por el productor o el consumidor.

Dado que en este caso hay condiciones de competencia, se debe tener un método que utiliza `synchronized` y al que deben acceder los tres hilos para realizar sus correspondientes operaciones.

Por claridad, su programa debe borrar la pantalla cada que se modifique la pila por lo que se debe ver como en el siguiente video:

<https://youtu.be/7Be3HHqXdWI>

Envíe como comprobante de este ejercicio además del código un pequeño video donde se observe su funcionamiento.